

Understanding Your Application's Network

And how to optimize it

huangyanshuo@meican.com

Preamble

History of Networking Issue

- Concurrency & Network
- “Connection Timeout” / “Context Deadline Exceeded”
- Cross components:
 - Infrastructure (Availability Zone, Security Group, Firewall)
 - DNS
 - Proxy
 - Source & Target application
 - Etc...

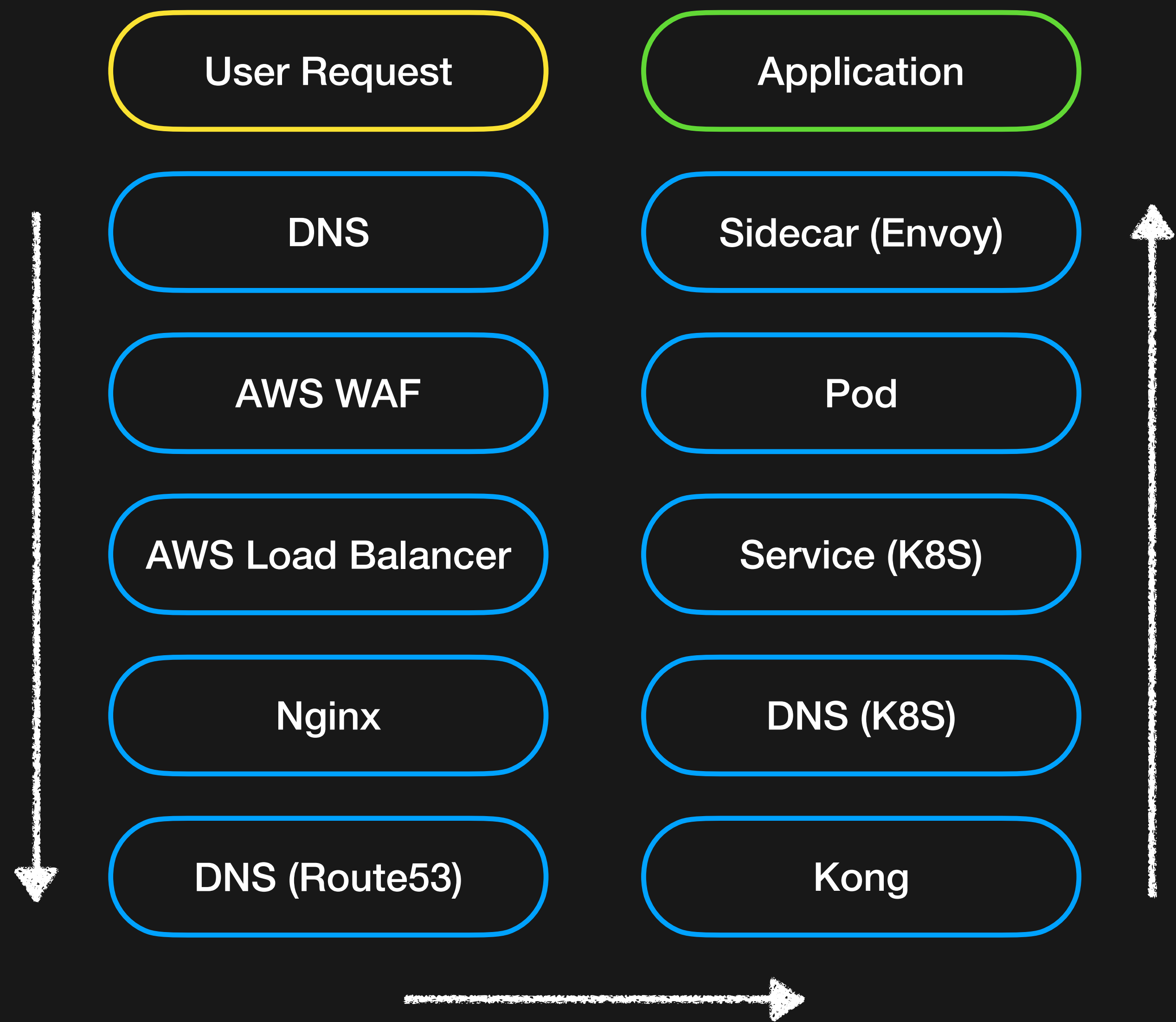
Agenda

- How a client request reaches an application ?
 - Traffic flow overview
- Inner cluster communication
 - Between applications
 - DNS
 - Istio Sidecar
- Inter cluster communication
 - Kong
- Only the basics, practices

From User to Application

Traffic Flow

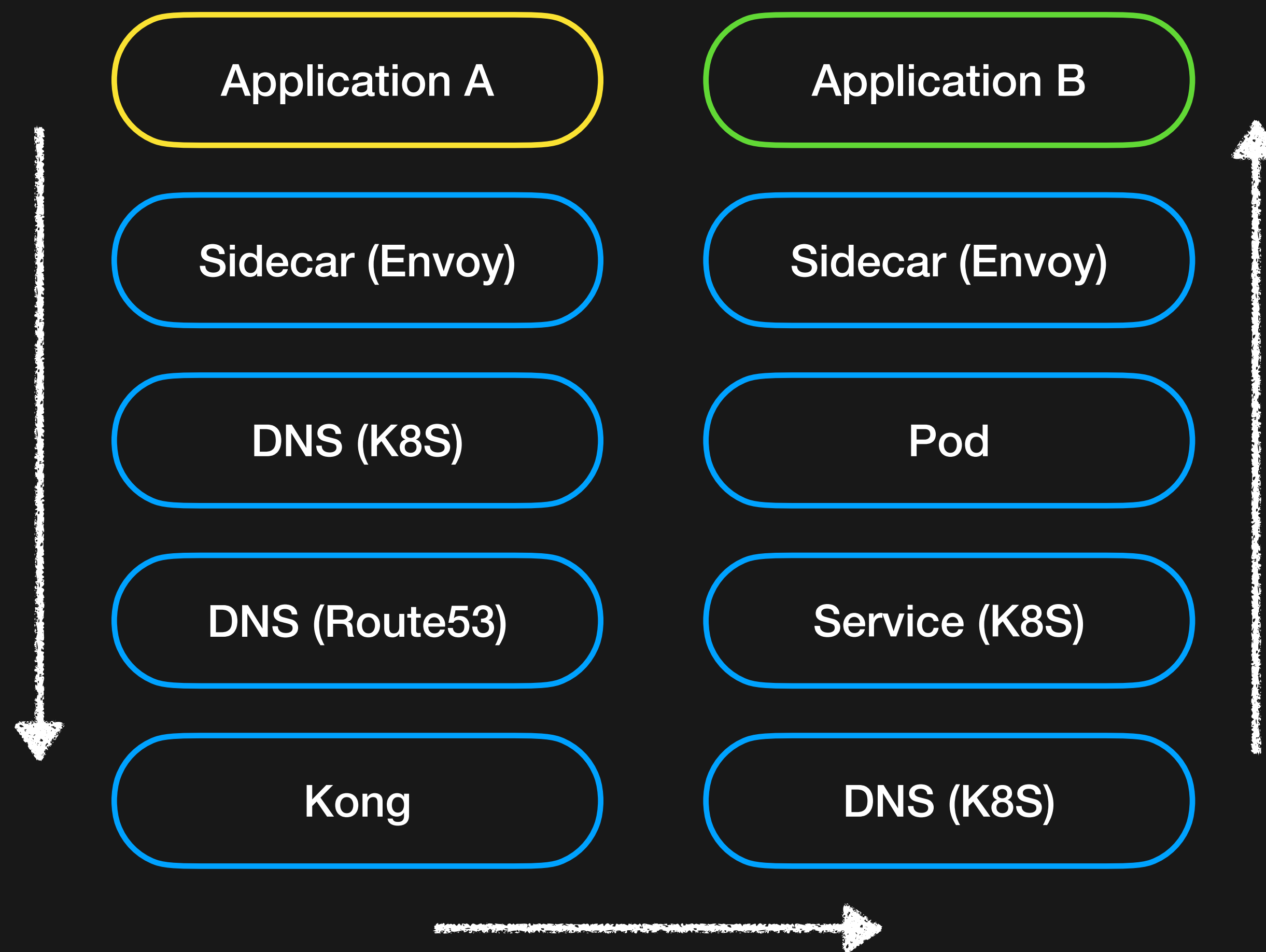
- Public internet
- Eg. sample-app.meican.com



Application Communication

Inter Cluster

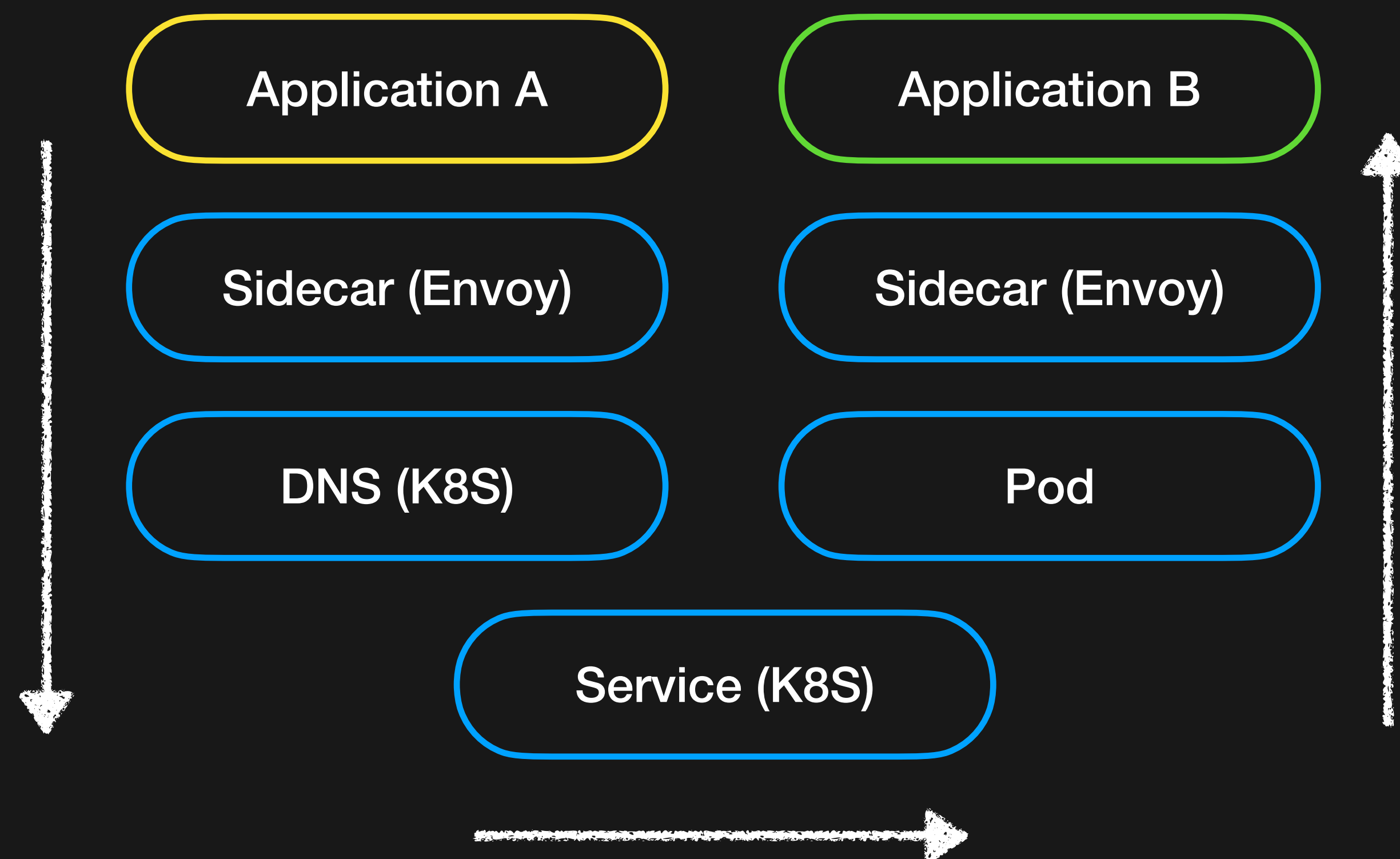
- From Cluster Fan1 -> Fan2
- Eg. sample.app.ntrnl-eks-fan2.meican



Application Communication

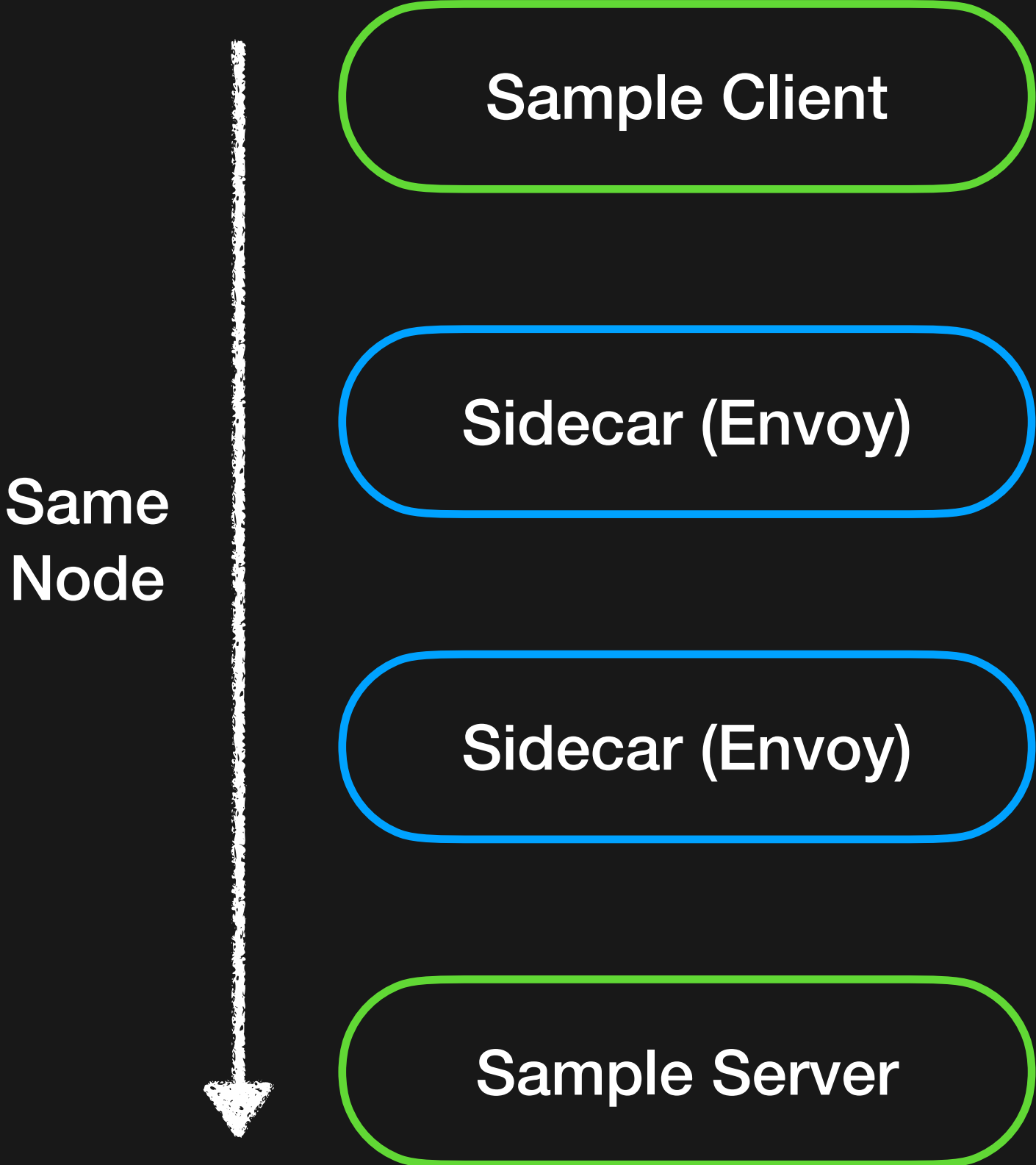
Intra Cluster

- Eg. `sample.app.svc.cluster.local`



Intra Cluster Traffic

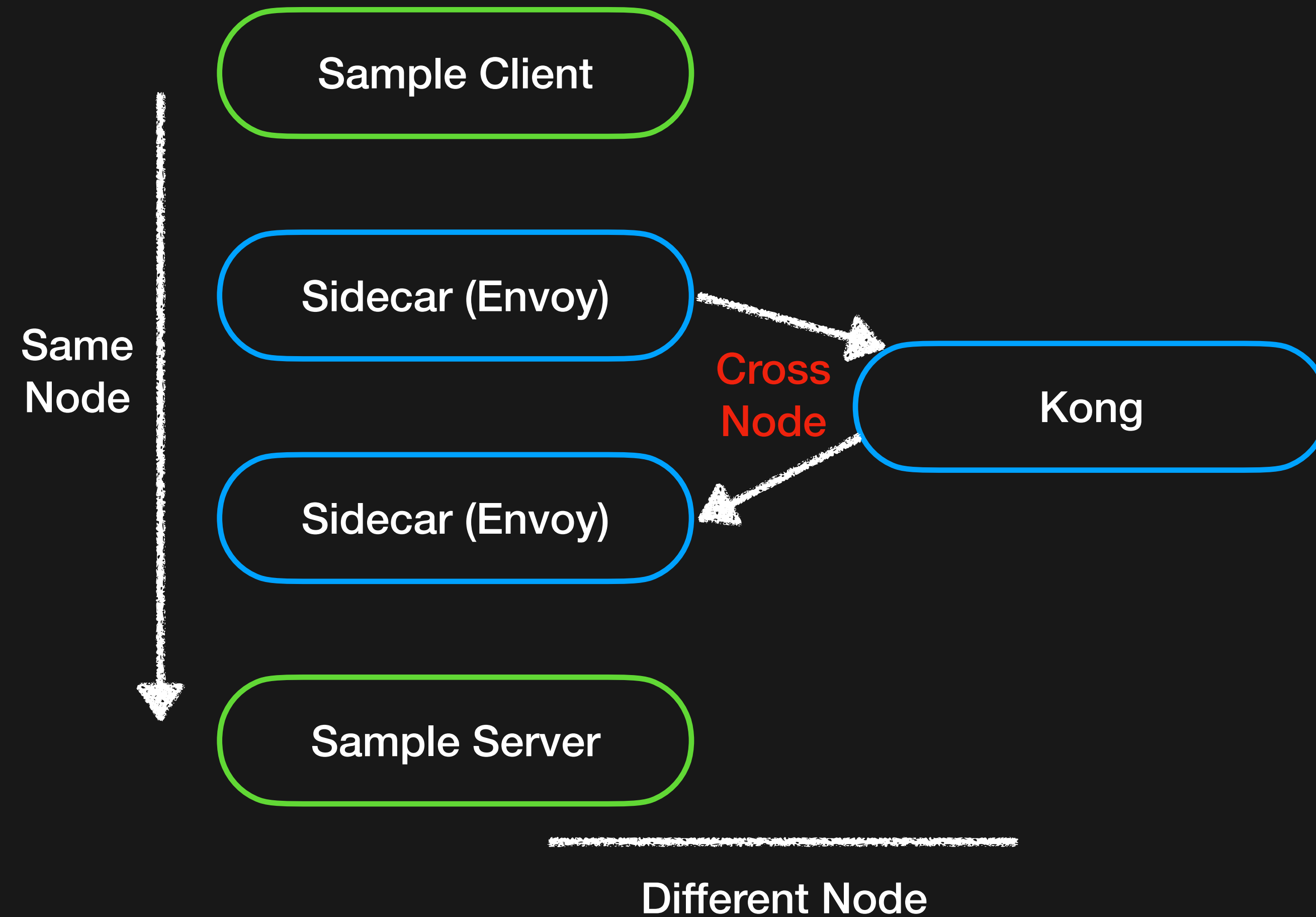
.svc.cluster.local



Time	172.24.128.23	172.24.144.189
0.857016	39938 → 8080 [SYN] Seq=0 Win=63680 L...	8080
0.857214	8080 → 39938 [SYN,ACK] Seq=0 Ack=1 Wi...	8080
0.857872	39938 → 8080 [ACK] Seq=1 Ack=1 Win=63...	8080
0.857985	39938 → 8080 GET /call HTTP/1.1	8080
0.858003	8080 → 39938 [ACK] Seq=1 Ack=87 Win=6...	8080
0.882534	39938 → 8080 HTTP/1.1 200 OK (text/plain)	8080
0.883191	39938 → 8080 [ACK] Seq=87 Ack=499 Win...	8080
0.883325	39938 → 8080 [FIN,ACK] Seq=87 Ack=499...	8080
0.883370	8080 → 39938 [FIN,ACK] Seq=499 Ack=88...	8080
0.884041	39938 → 8080 [ACK] Seq=88 Ack=500 Wi...	8080

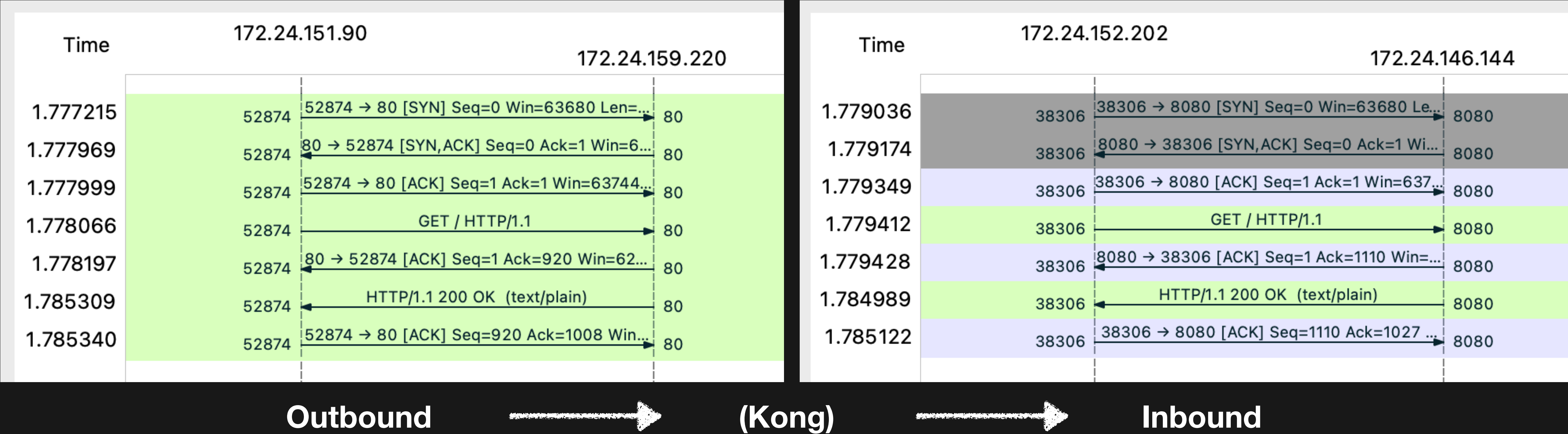
Inter Cluster Traffic

.ntrnl-eks-fan.meican



Inter Cluster Traffic

.ntrnl-eks-fan.meican



Comparison Between Inter and Intra Traffic Benchmarking

- Observed ~10% drawdown
- “.ntrnl-eks-fan.meican” adds at least 1 hop (one-way)

```
wrk -t2 -c10 -d30s http://client-service.testing:8080/call?keepalive=false
Running 30s test @ http://client-service.testing:8080/call?keepalive=false
2 threads and 10 connections
Thread Stats   Avg      Stdev     Max    +/-  Stdev
  Latency    13.01ms    5.63ms   83.15ms   83.05%
  Req/Sec    393.55     64.52   505.00    67.33%
23521 requests in 30.02s, 11.16MB read
Requests/sec: 783.49
Transfer/sec: 380.80KB
```

.svc.cluster.local

```
wrk -t2 -c10 -d30s http://client-service.testing:8080/call?keepalive=false
Running 30s test @ http://client-service.testing:8080/call?keepalive=false
2 threads and 10 connections
Thread Stats   Avg      Stdev     Max    +/-  Stdev
  Latency    15.33ms    8.21ms  143.22ms   89.42%
  Req/Sec    331.73     60.87   450.00    68.00%
19824 requests in 30.02s, 9.42MB read
Socket errors: connect 0, read 0, write 0, timeout 3
Requests/sec: 660.45
Transfer/sec: 321.26KB
```

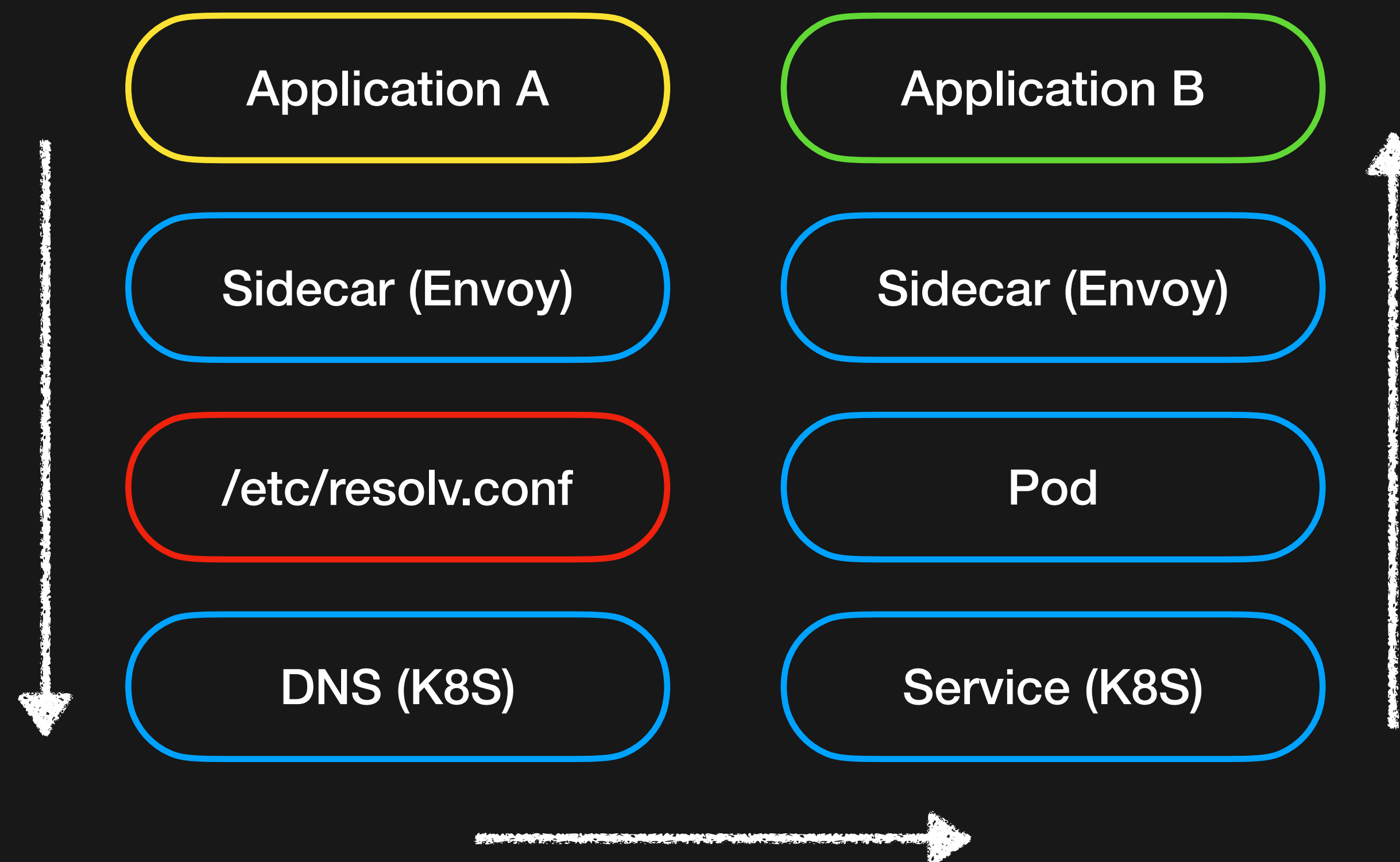
.ntrnl-eks-fan.meican

Optimization #1: Use Inner Cluster Address

If Possible

- Same cluster: prefer `.svc.cluster.local` instead of `.ntrnl-eks-fan.meican`
- Different cluster: `.ntrnl-eks-fan.meican`
- Potential optimization

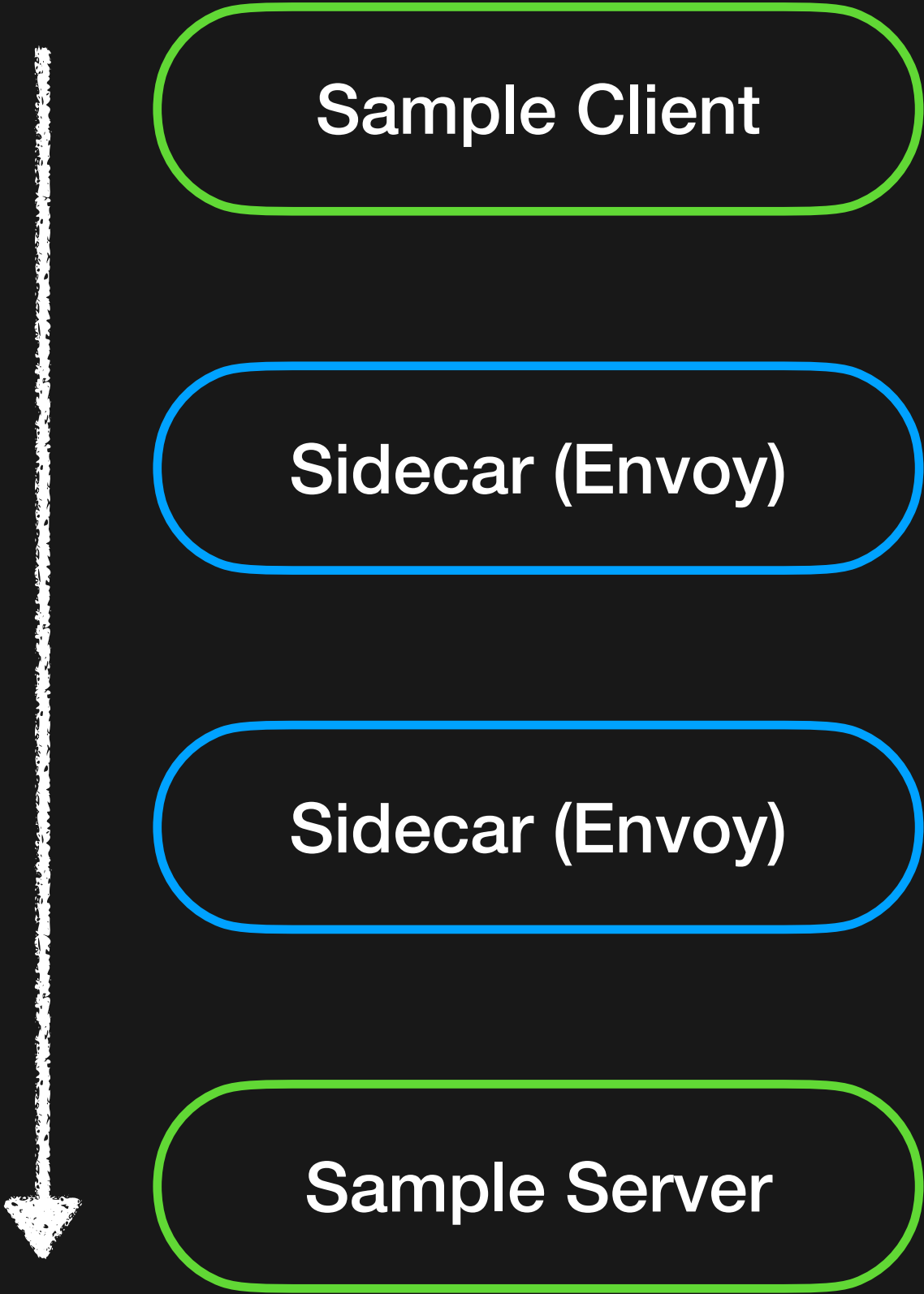
Pod DNS Resolver



Pod DNS Resolver

Too Many DNS Queries

Same
Node



Time	172.24.144.189	172.24.151.39	172.24.129.95
0.859270	53977	Standard query 0x8ebe AAAA server-servic...	53
0.859271	32925	Standard query 0xd169 A server-service1.te...	53
0.859829	32925	Standard query response 0xd169 No such n...	53
0.859960	53977	Standard query response 0x8ebe No such n...	53
0.860146	57892	Standard query 0x3244 A server-service1.testing.svc.cluster.local.svc.cluster.local OPT	53
0.860159	45971	Standard query 0xc4e0 AAAA server-servic...	53
0.860580	45971	Standard query response 0xc4e0 No such n...	53
0.861150	57892	Standard query response 0x3244 No such name A server-service1.testing.svc.cluster.loca...	53
0.861293	53458	Standard query 0x187f AAAA server-service1.testing.svc.cluster.local.cluster.local OPT	53
0.861368	55769	Standard query 0xe172 A server-service1.te...	53
0.861841	55769	Standard query response 0xe172 No such n...	53
0.862227	53458	Standard query response 0x187f No such name AAAA server-service1.testing.svc.cluster.l...	53
0.862337	40724	Standard query 0xcae8 AAAA server-service1.testing.svc.cluster.local.cn-northwest-1.co...	53
0.862407	35362	Standard query 0x438f A server-service1.testing.svc.cluster.local.cn-northwest-1.comput...	53
0.864416	35362	Standard query response 0x438f No such name A server-service1.testing.svc.cluster.local...	53
0.864545	40724	Standard query response 0xcae8 No such name AAAA server-service1.testing.svc.cluster.l...	53
0.864681	51218	Standard query 0xb245 AAAA server-servic...	53
0.864767	59671	Standard query 0x91f8 A server-service1.te...	53
0.865187	59671	Standard query response 0x91f8 A server-s...	53
	51218	Standard query response 0xb245 AAAA ser...	53

Pod DNS Resolver

Wasted DNS Queries

- Query “server-serivice1.testing.svc.cluster.local”
- Local DNS Resolver runs for both A and AAAA record
 - \$ADDR + .testing.svc.cluster.local
 - \$ADDR + .svc.cluster.local
 - \$ADDR + .cluster.local
 - \$ADDR + .cn-northwest-1.compute.internal
- All “No such name”
- \$ADDR + nothing -> type A result 10.100.124.158
- 8(9) DNS queries wasted, why ?

```

  Answers
  server-service1.testing.svc.cluster.local: type A,
    Name: server-service1.testing.svc.cluster.local
    Type: A (Host Address) (1)
    Class: IN (0x0001)
    Time to live: 5
    Data length: 4
    Address: 10.100.124.158

```

Pod DNS Resolver

Why Wasting DNS Queries?

- When requesting, valid addresses are:
 - Same namespace: server-service
 - Same cluster: server-service.testing
 - FQDN: server-service.testing.svc.cluster.local

Optimization: Pod DNS Resolver

Explain the /etc/resolv.conf

```
~ # cat /etc/resolv.conf
search testing.svc.cluster.local svc.cluster.local cluster.local cn-northwest-1.compute.internal
nameserver 10.100.0.10
options ndots:5
```

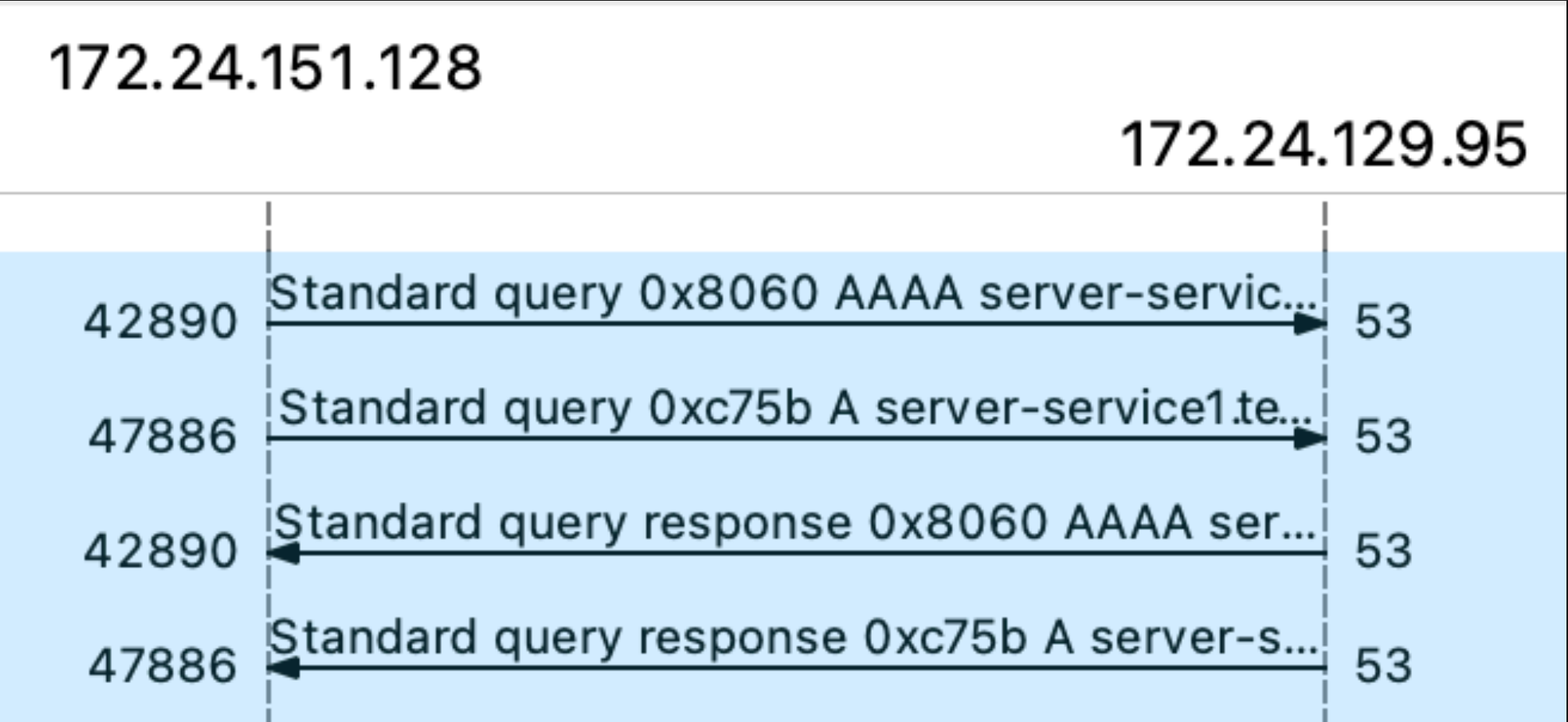
- Search paths: <name>.<suffix>
- Name server: CoreDNS inside cluster
- ndots:
 - <5 dots, search full paths
 - >= 5 dots, direct search
- server-service1.testing.svc.cluster.local <- 4 dots, full paths search

Optimization #2: Pod DNS Resolver

Change /etc/resolv.conf

```
spec:
  replicas: 1
  selector:
    matchLabels:
      app: test-client
  template:
    metadata:
      labels:
        app: test-client
        sidecar.istio.io/inject: "true"
    spec:
      dnsConfig:
        options:
          - name: ndots
            value: "3"
```

Via pod.spec.dnsConfig



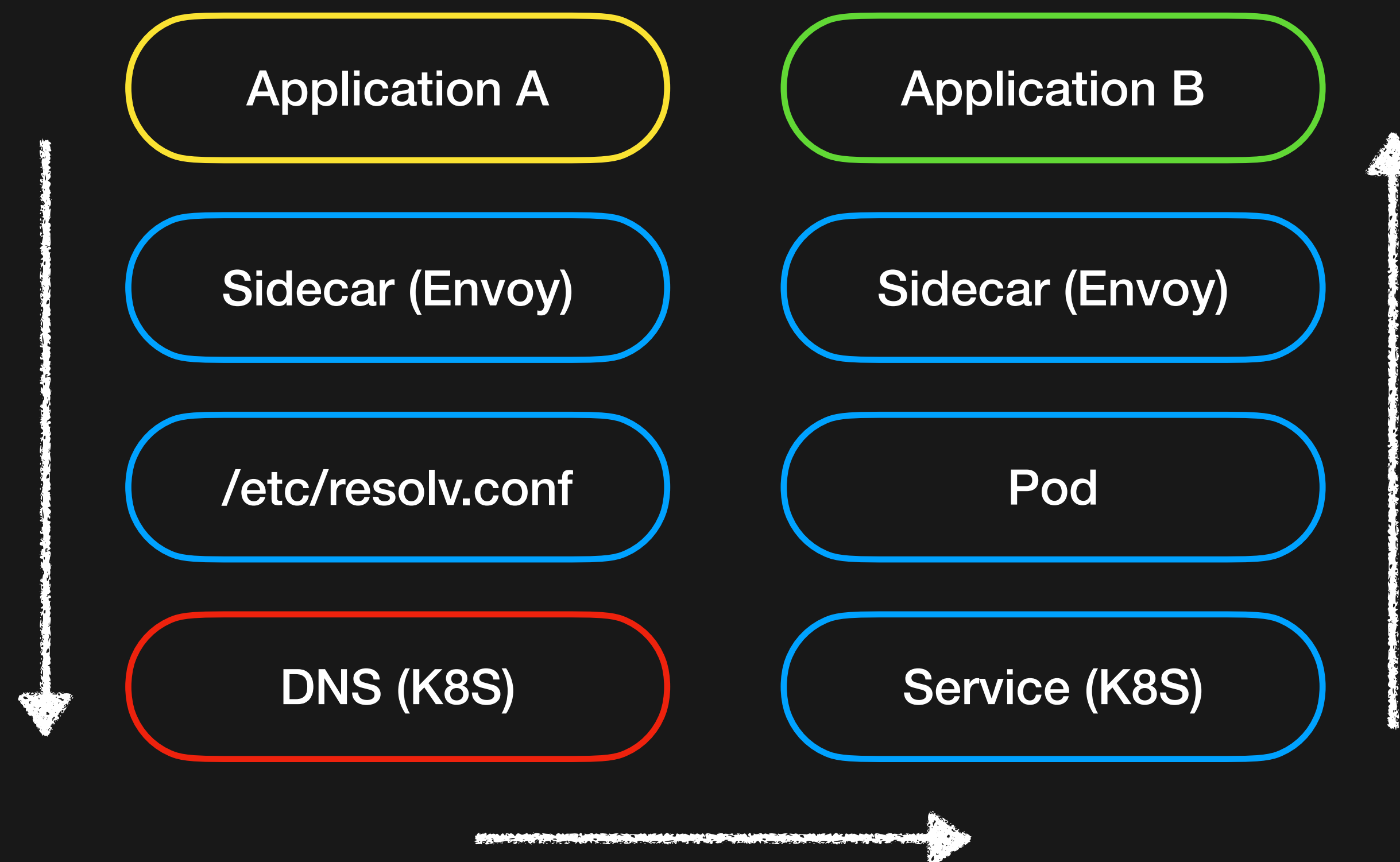
Direct result

Optimization #2: Pod DNS Resolver

Summary

- Safe changing “ndots”, save DNS queries
- -200ms avg.
- Most languages caches the DNS result

Kubernetes DNS



Kubernetes DNS

Rewrite record

- What if you insist on using `.ntrnl-eks-fan.meican` ?
- Optimization we've done for Pulsar:
 - Requesting `pulsar-broker.pulsar.ntrnl-eks-fan.meican`

Kubernetes DNS

Optimization we've done for Pulsar

Requesting pulsar-broker.pulsar.ntrnl-eks-fan.meican

Domain Name System (response)

Transaction ID: 0xc34a
 > Flags: 0x8500 Standard query response, No error
 Questions: 1
 Answer RRs: 3
 Authority RRs: 0
 Additional RRs: 1
 > Queries
 > Answers

> pulsar-broker.pulsar.ntrnl-eks-fan.meican: type A, class IN, addr 172.24.148.231
 > pulsar-broker.pulsar.ntrnl-eks-fan.meican: type A, class IN, addr 172.24.145.68
 > pulsar-broker.pulsar.ntrnl-eks-fan.meican: type A, class IN, addr 172.24.157.230

DNS Answers

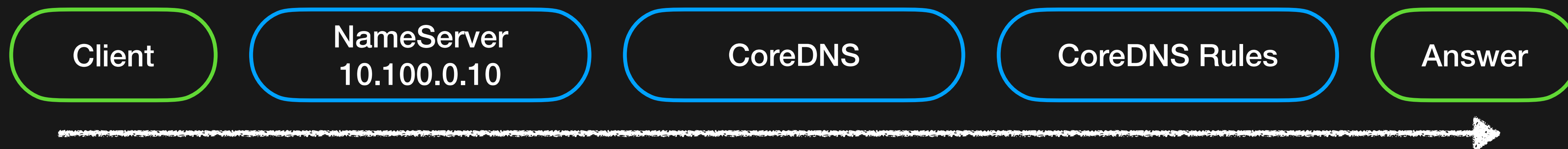
\$ k -n pulsar get pod -l component=broker -o wide

NAME	READY	STATUS	RESTARTS	AGE	IP
pulsar-broker-1b-0	1/1	Running	0	13d	172.24.148.231
pulsar-broker-1b-1	1/1	Running	0	14d	172.24.157.230
pulsar-broker-1b-2	1/1	Running	0	2d7h	172.24.145.68

Same IPs

Optimization #3: Kubernetes DNS

Rewrite record



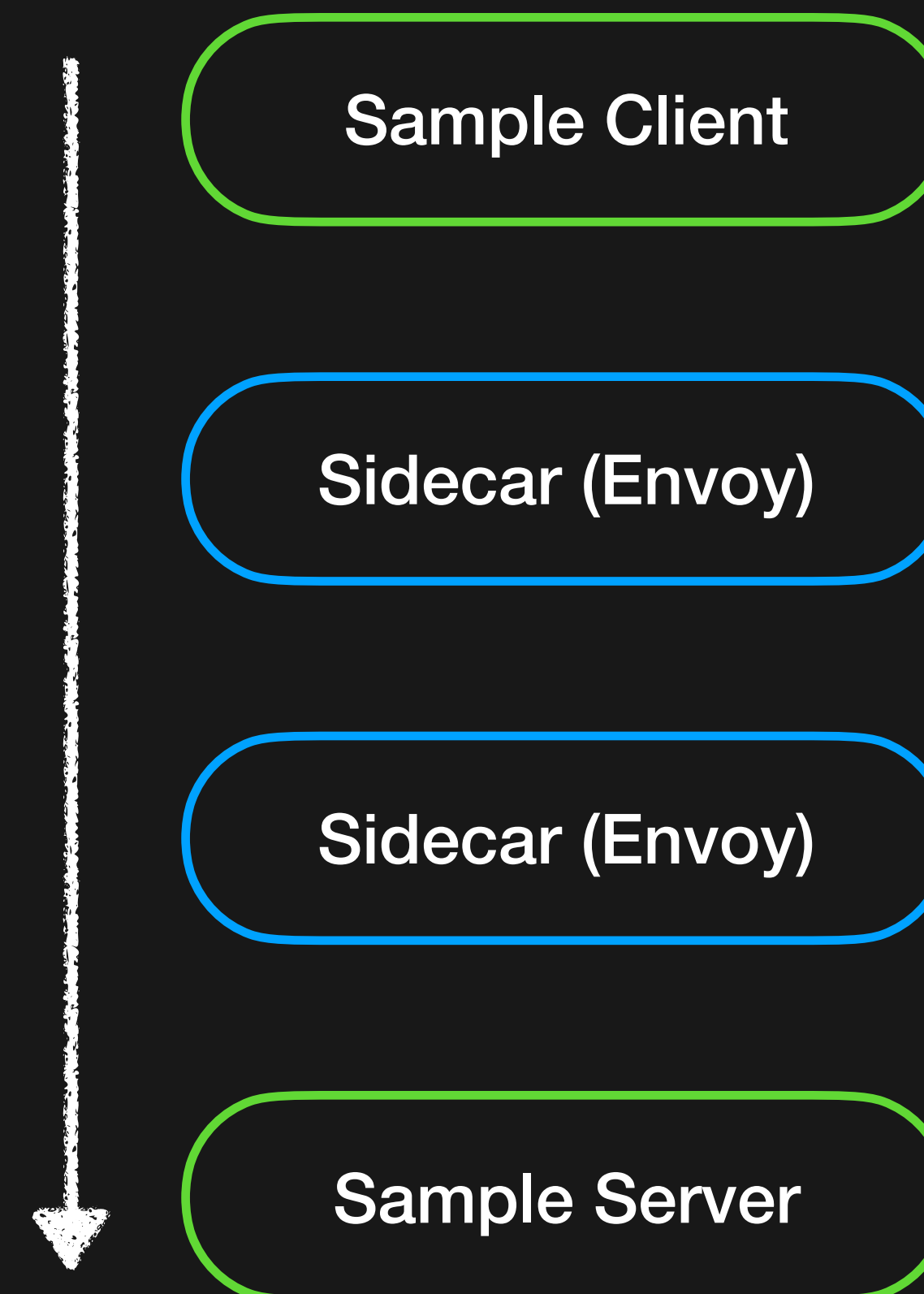
```

pulsar-broker.pulsar.ntrnl-eks-fan.meican {
  forward . 127.0.0.1
  rewrite stop {
    name regex pulsar-broker.pulsar.ntrnl-eks-fan.meican pulsar-broker.pulsar.svc.cluster.local
    answer name pulsar-broker.pulsar.svc.cluster.local pulsar-broker.pulsar.ntrnl-eks-fan.meican
  }
}

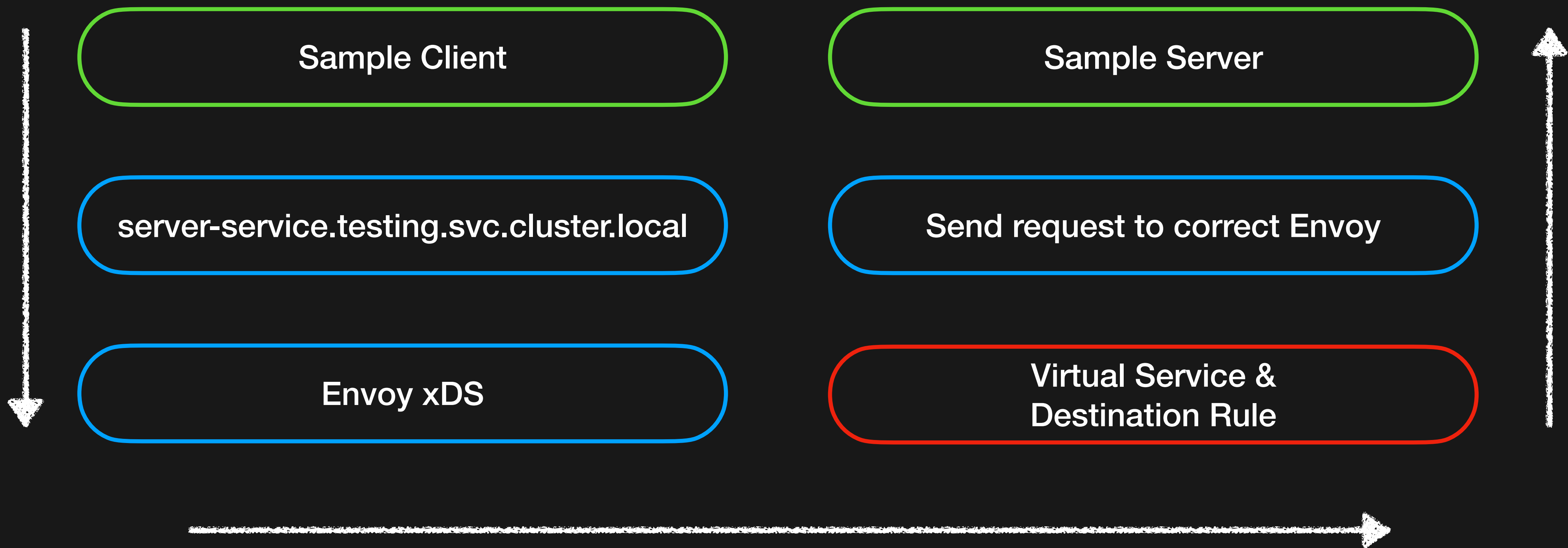
```

Credit to @songchao

Istio Sidecar Traffic

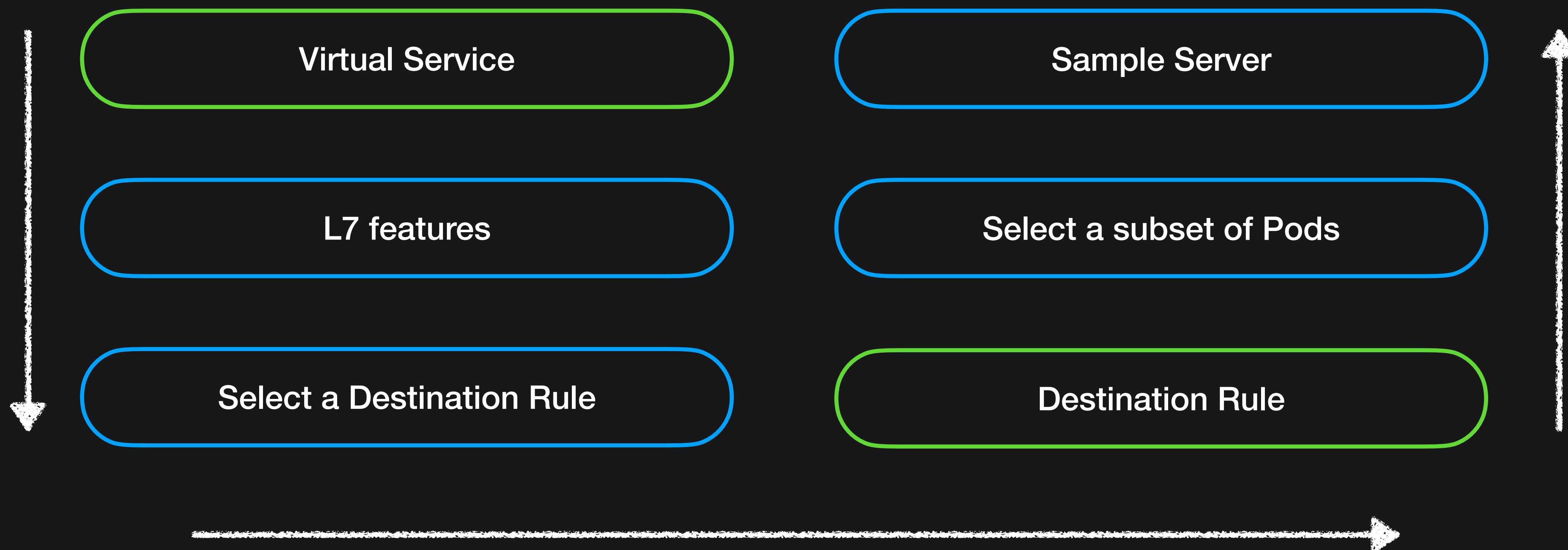


Istio Sidecar Traffic Overview



Istio Sidecar Traffic

Virtual Service and Destination Rule



Istio Sidecar Traffic

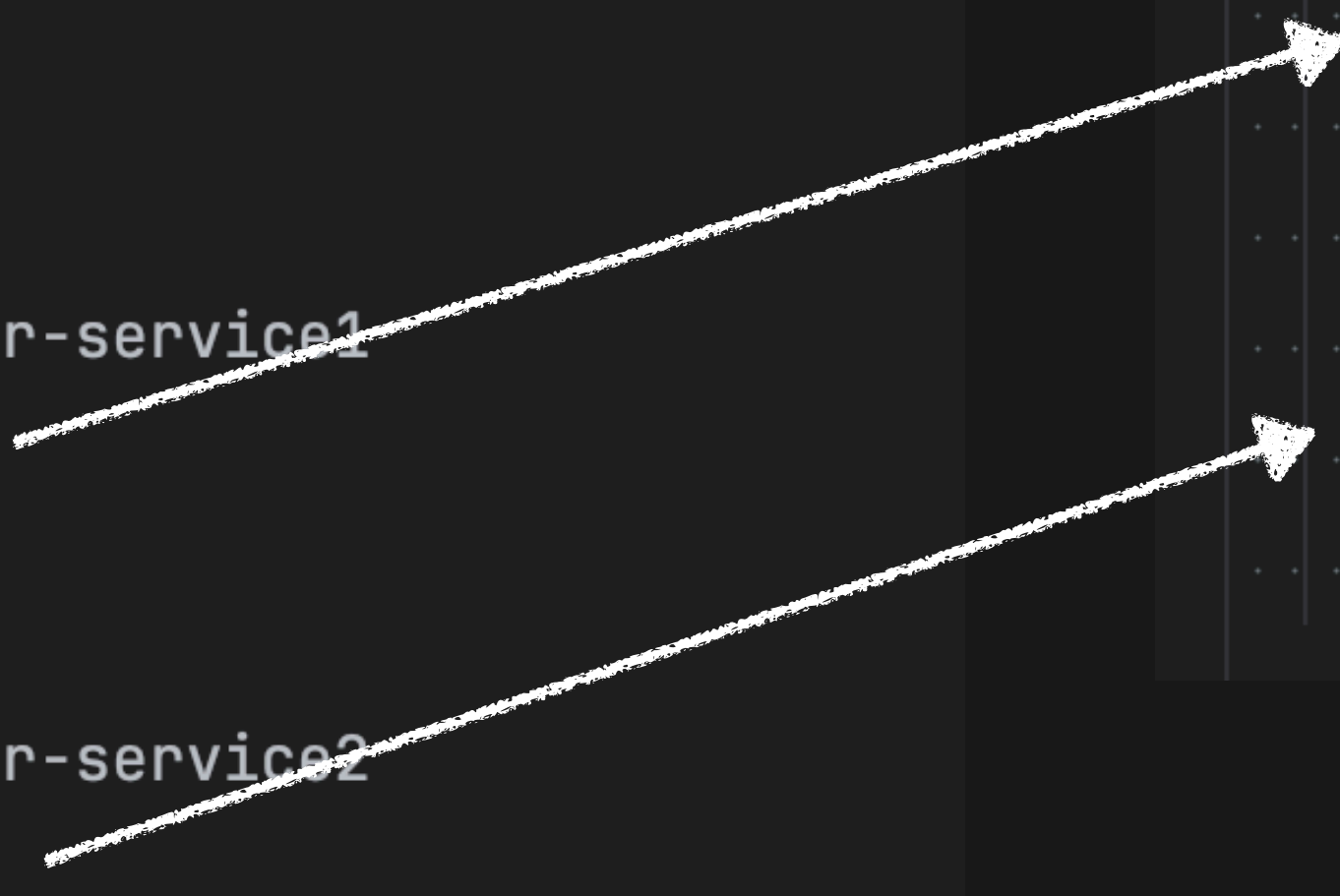
Virtual Service and Destination Rule

```
kind: VirtualService
metadata:
  labels:
    app: test-server
  name: test-server
  namespace: testing
spec:
  hosts:
    - server-service.testing.svc.cluster.local
  http:
    - name: primary
      route:
        - destination:
            host: server-service1
            subset: v1
            weight: 50
        - destination:
            host: server-service2
            subset: v2
            weight: 50
```

Call

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: server-service
  namespace: testing
spec:
  host: server-service.testing.svc.cluster.local
  subsets:
    - name: v1
      labels:
        version: v1
    - name: v2
      labels:
        version: v2
```

Match



Istio Sidecar Traffic

Virtual Service and Destination Rule

```

apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: server-service
  namespace: testing
spec:
  host: server-service.testing.svc.cluster.local
  subsets:
    - name: v1
      labels:
        version: v1
    - name: v2
      labels:
        version: v2

```

Match

```

apiVersion: v1
kind: Service
metadata:
  name: server-service1
  namespace: testing
  labels:
    version: v1
spec:
  selector:
    app: test-server
    version: v1

```

```

apiVersion: v1
kind: Service
metadata:
  name: server-service2
  namespace: testing
  labels:
    version: v2
spec:
  selector:
    app: test-server
    version: v2

```

Select pods

- Very complicated:
 - <https://gitlab.planetmeican.com/meican-cd/issues>
- If only using Envoy communications, no need to define:
 - Service: server-service.testing.svc.cluster.local
- Need to define:
 - Virtual Service & Destination Rule
 - Service (in order to select pods):
 - server-service1.testing.svc.cluster.local
 - server-service2.testing.svc.cluster.local

meican-cd / issues / Issues / New

Issue

Description

Init

Normal text

B

I

≡

</>

≡

≡

≡

+

改模块作用于 sandbox环境 terraform & argo 项目初始化

1. <https://gitlab.planetmeican.com/meican-cd/terraform-sandbox>

2. <https://gitlab.planetmeican.com/meican-cd/argocd-sandbox>

-----Please Do Not Edit This Line, all info For Terraform and Argo init-----

后台脚本会帮忙创建关联资源, 请更新如下的 json 内容。

步骤1: Terraform 管理信息整理

{

"GroupName": "meican-cd",

"ProjectName": "issues",

"Types": ["web", "grpc"]

}

Istio Virtual Service

L7 Features

- Defined by the callee side
- Note: timeout will override app's timeout configuration

```
spec:
  hosts:
  - server-service.testing.svc.cluster.local
  http:
  - name: primary
    timeout: 10s
    retries:
      attempts: 3
      retryOn: connect-failure,refused-stream,unavailable
```

Istio Destination Rule

Retry, Timeout and etc...

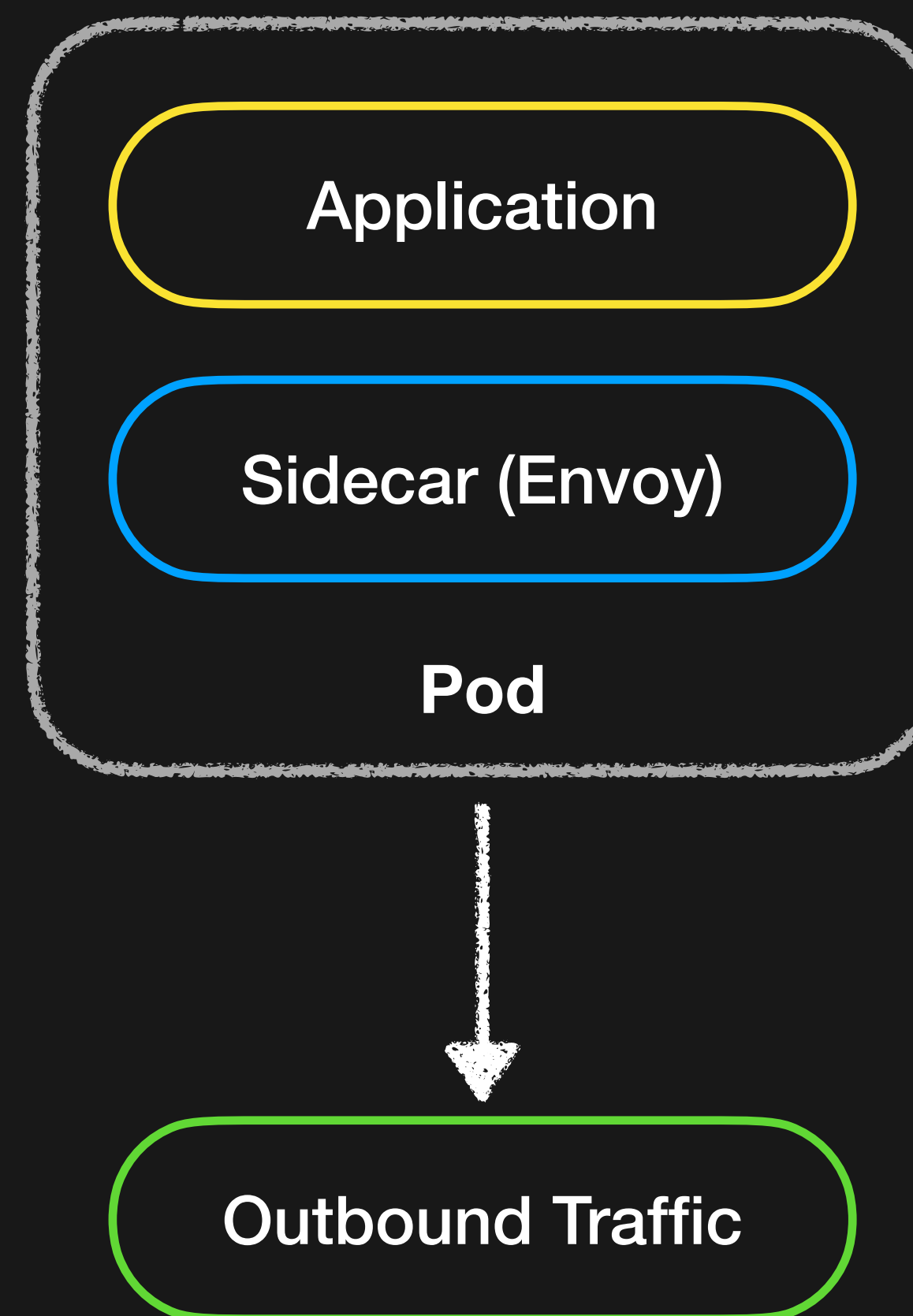
- Defined for the caller
- Load balancer
- Connection Pool
- Outlier Detection
- Etc...

```

apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: server-service
  namespace: testing
spec:
  host: server-service.testing.svc.cluster.local
  trafficPolicy:
    loadBalancer:
      simple: LEAST_REQUEST
      warmup:
        duration: 10s
    connectionPool:
      tcp:
        maxConnections: 99
      http:
        http1MaxPendingRequests: 1024
        maxRequestsPerConnection: 33
    outlierDetection:
      consecutive5xxErrors: 100
      interval: 1m
      baseEjectionTime: 5m
  
```


Istio Connection Pooling

How Envoy takes over the traffic ?



Istio Connection Pooling

What happens to application's traffic ?

- Envoy will take over the requests
- Istio-init on pod init phase
- Rewrite Pod's IPtables

```
*nat
:PREROUTING ACCEPT [0:0]
:INPUT ACCEPT [0:0]
:OUTPUT ACCEPT [0:0]
:POSTROUTING ACCEPT [0:0]
:ISTIO_INBOUND - [0:0]
:ISTIO_IN_REDIRECT - [0:0]
:ISTIO_OUTPUT - [0:0]
:ISTIO_REDIRECT - [0:0]
-A PREROUTING -p tcp -j ISTIO_INBOUND
-A OUTPUT -p tcp -j ISTIO_OUTPUT
```


Istio Connection Pooling

Comparison Connection Pooling Performance

- Mock 20ms response lag
- 2 threads, 50 connections
- Connection pool size: 1

```
trafficPolicy:
  connectionPool:
    tcp:
      maxConnections: 1
    http:
      maxRequestsPerConnection: 1
```

```
/app # wrk -t2 -c50 -d30s http://client-service.testing:8080/call
Running 30s test @ http://client-service.testing:8080/call
2 threads and 50 connections
Thread Stats   Avg      Stdev     Max    +/-  Stdev
  Latency   564.21ms  100.48ms  686.44ms   87.51%
  Req/Sec    43.91    11.28    80.00   63.83%
2635 requests in 30.02s, 1.25MB read
Requests/sec:    87.76
Transfer/sec:    42.76KB
```


Optimization #4: Istio Connection Pooling

Comparison Connection Pooling Performance

- Mock 20ms response lag
- 2 threads, 50 connections
- Default settings
- 80 rps vs 900 rps
- Basically no need to optimize, unless you want to protect your application

```
"circuit_breakers": {
  "thresholds": [
    {
      "max_connections": 4294967295,
      "max_pending_requests": 4294967295,
      "max_requests": 4294967295,
      "max_retries": 4294967295,
      "track_remaining": true
    }
  ]
}
```

```
/app # wrk -t2 -c50 -d30s http://client-service.testing:8080/call
Running 30s test @ http://client-service.testing:8080/call
2 threads and 50 connections
Thread Stats   Avg      Stdev     Max    +/-  Stdev
  Latency    56.59ms   25.81ms  255.97ms   82.15%
  Req/Sec   454.08   152.56   720.00    59.03%
27053 requests in 30.02s, 12.85MB read
Requests/sec:   901.19
Transfer/sec:   438.23KB
```

Check “Advanced gRPC”

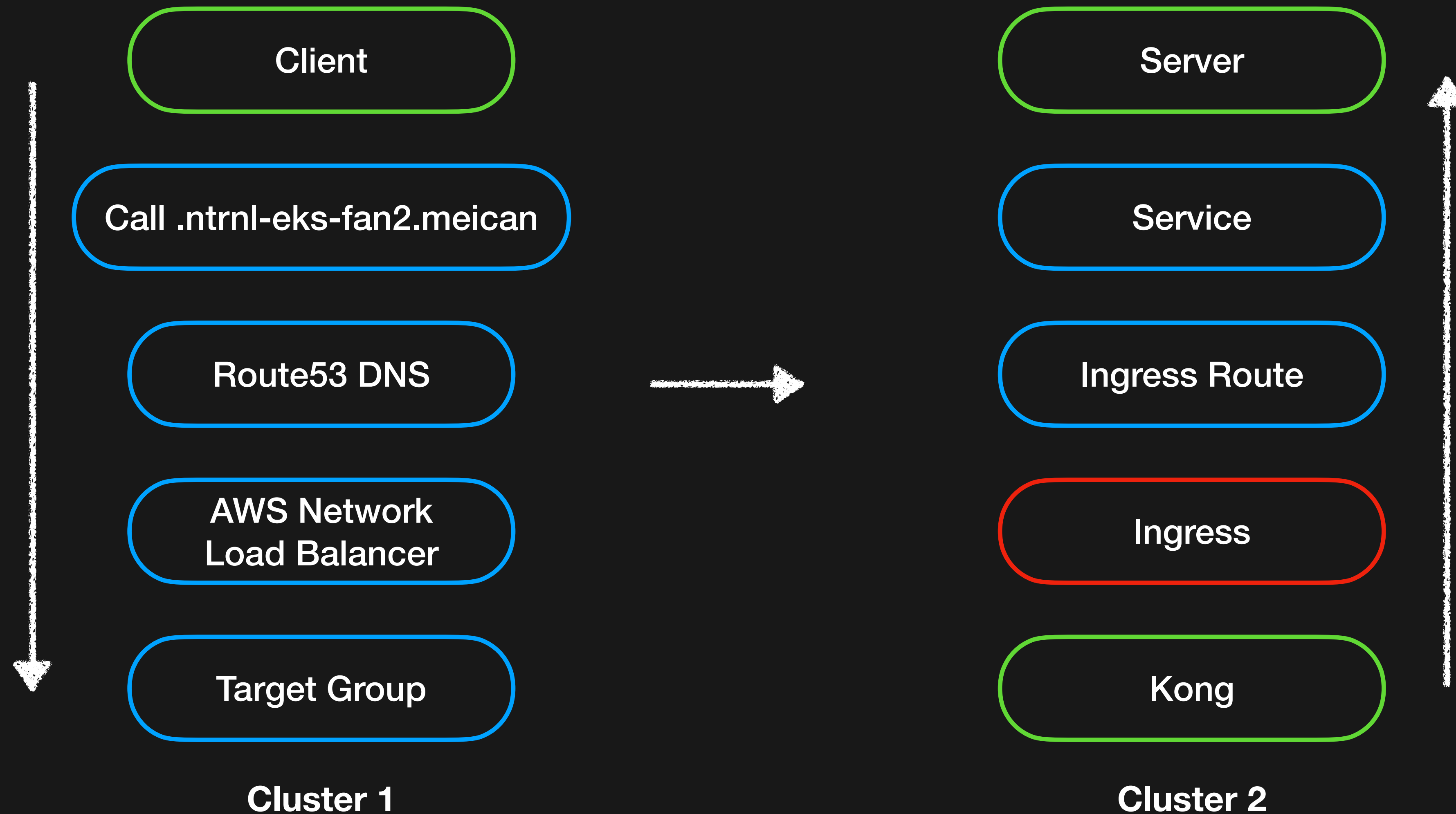
Optimization #5: Istio Connection Pooling

Rolling Update with Kong

Why client still encounter 503 when rolling update or HPA down sizing?
 Don't we already have ArgoCD rolling update ?

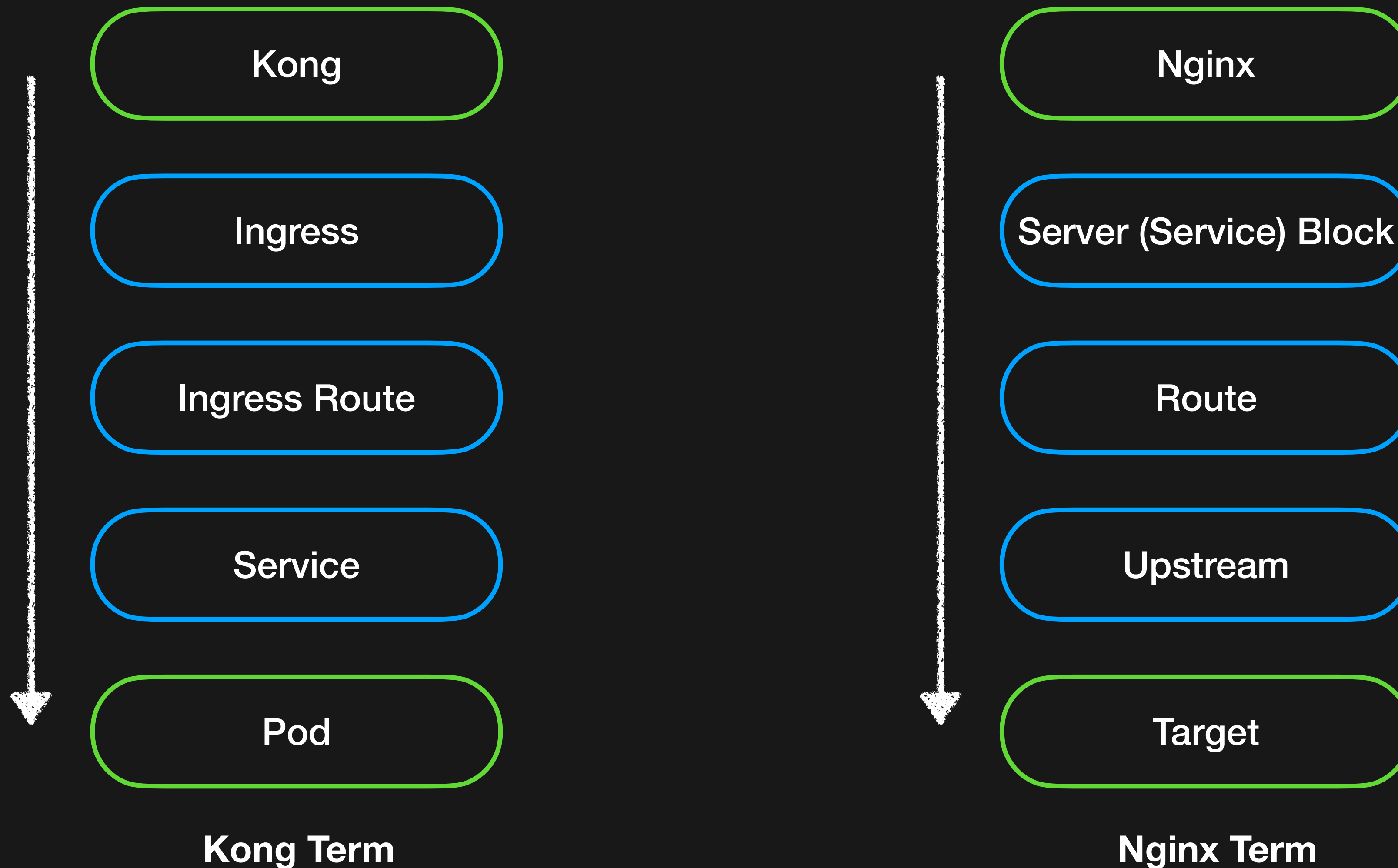
Rolling Update with Kong

Traffic Overview



Rolling Update with Kong

Terminology



Rolling Update with Kong

Manifest

```

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: server-service1-kong1
  namespace: testing
  annotations:
    kubernetes.io/ingress.class: "kong1"
    konghq.com/protocols: "http"
spec:
  rules:
    - host: server-service.testing.ntrnl-eks-fan.meican
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: server-service
                port:
                  name: http

```

```

apiVersion: v1
kind: Service
metadata:
  name: server-service
  namespace: testing
  labels:
    version: v1
spec:
  selector:
    app: test-server
  ports:
    - port: 8080
      targetPort: 8080
      name: http

```

```

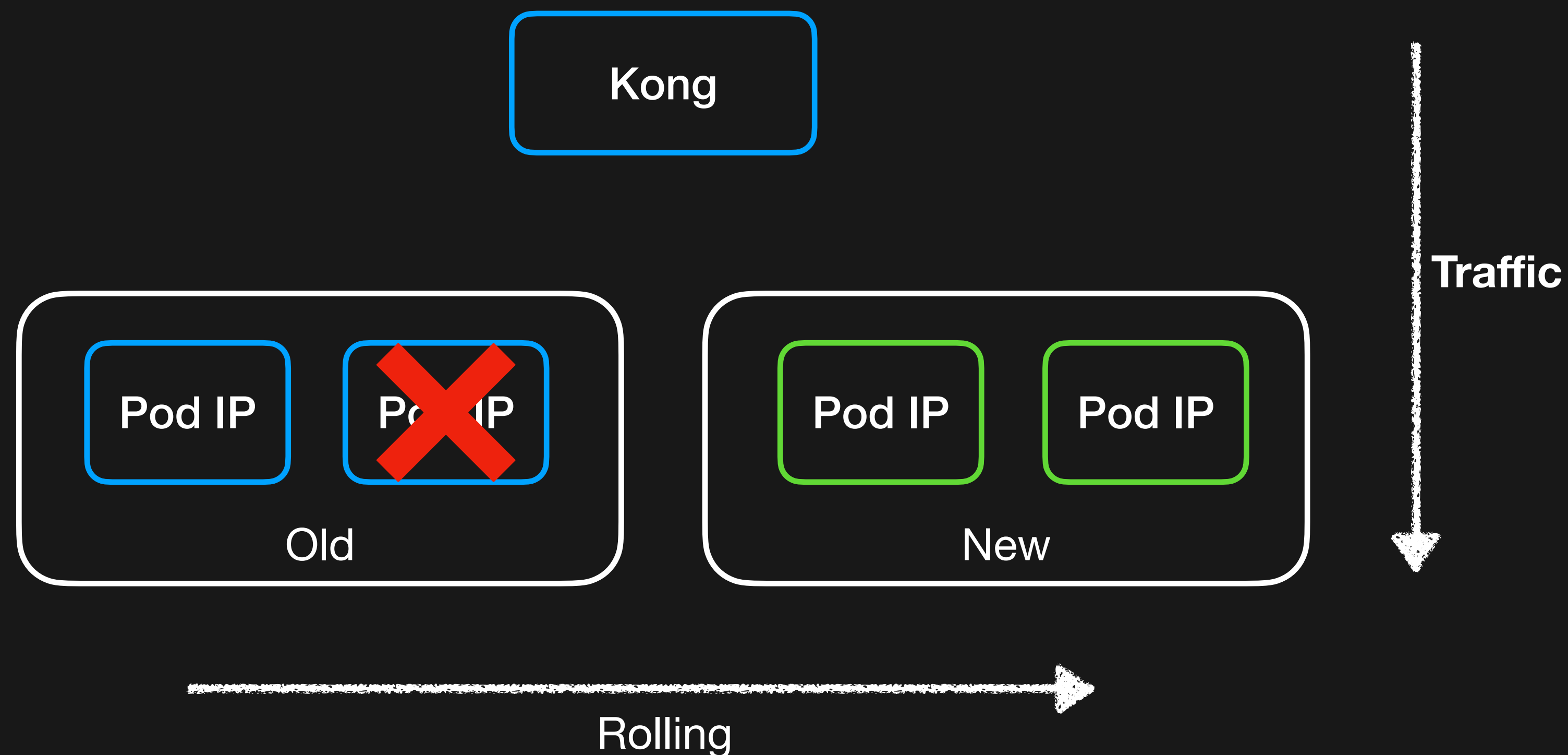
apiVersion: apps/v1
kind: Deployment
metadata:
  name: test-server1
  namespace: testing
spec:
  replicas: 1
  selector:
    matchLabels:
      app: test-server
      version: "v1"
  template:
    metadata:
      labels:
        app: test-server
        version: "v1"
        sidecar.istio.io/inject: "true"
    spec:
      nodeName: ip-172-24-145-105.cn-n

```


Rolling Update with Kong

Root Cause

- Though it seems that Kong searches service
- But Kong saves Pod IPs
- Updating IPs has latency



```
1430 requests in 30.03s, 697.42KB read
Socket errors: connect 0, read 0, write 0, timeout 2
Non-2xx or 3xx responses: 2
```

Optimization #6: Rolling Update with Kong

Enable service-upstream

```

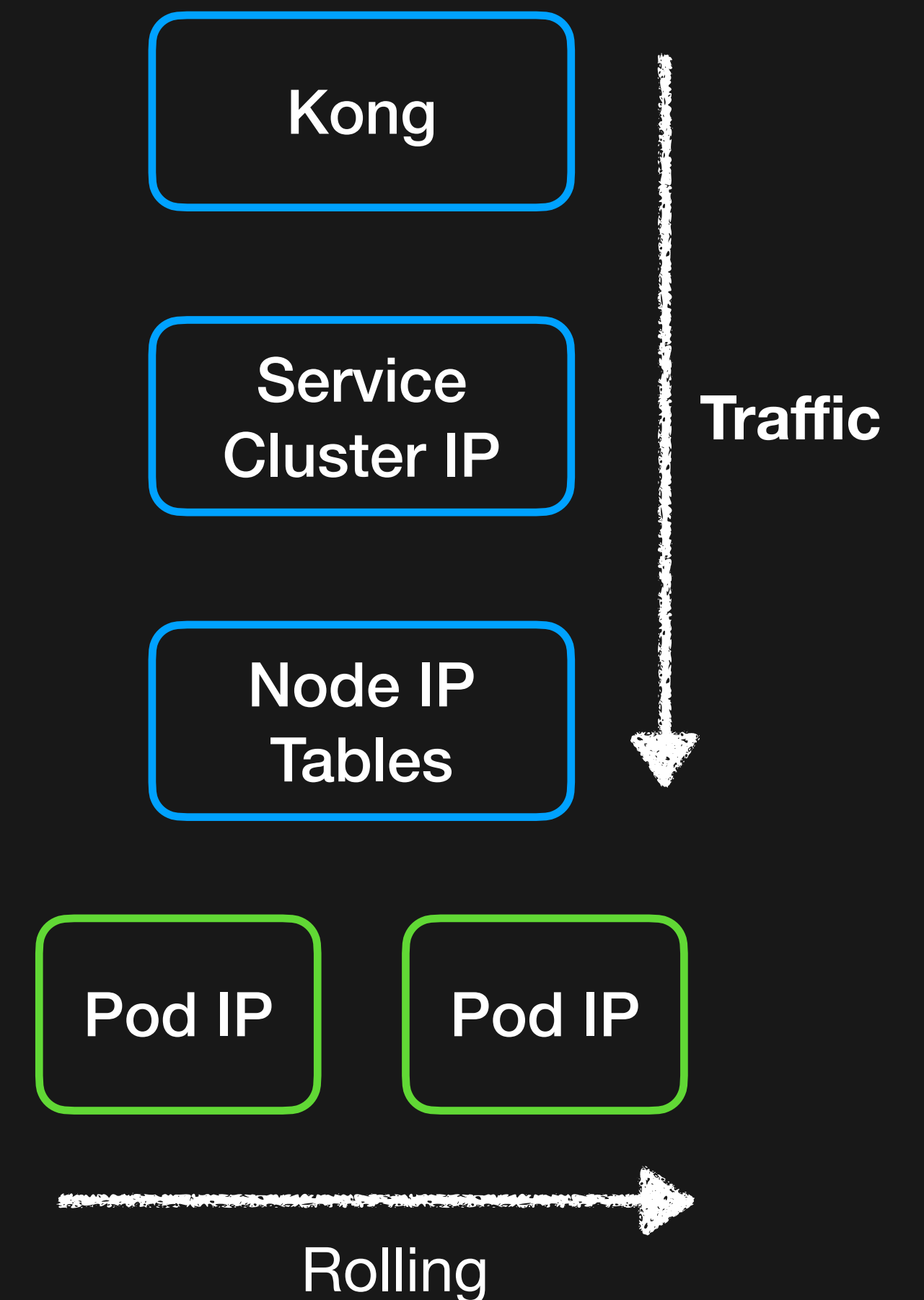
annotations:
  ingress.kubernetes.io/service-upstream: "true"
  
```

On Service, use kube-proxy IPtables

```

-s 172.24.159.105/32 -j KUBE-MARK-MASQ
-p tcp -m tcp -j DNAT --to-destination 172.24.159.105:8080
  
```

IPtables Rules to Pod IP



Optimization #6: Rolling Update with Kong

Comparison

```
Socket errors: connect 0, read 0, write 0, timeout 2
Non-2xx or 3xx responses: 2
```

Default IP mode

```
Socket errors: connect 0, read 0, write 0, timeout 2
```

Enable upstream-service

Optimization #6: Rolling Update with Kong

Enable service-upstream

- Kube-proxy use a dedicated listener for IP changes
 - On every node
 - With highest resource priority
- Less IP changing latency
- Less performance: -20% performance
 - Node resource
 - IPtables can be very long: 10k + rules

Optimization #7: Pod Pre-stop Hook

Rolling Update with Kong

- Pre-stop hook: terminating status
- When pod's in terminating status, Kong will remove it from IP map
- While the pod can still serve requests

```
resources:
  limits:
    cpu: "2000m"
    memory: "4Gi"
  requests:
    cpu: 50m
    memory: "32Mi"
lifecycle:
  preStop:
    exec:
      command:
        - "/bin/sh"
        - "-c"
        - |
          sleep 15
```

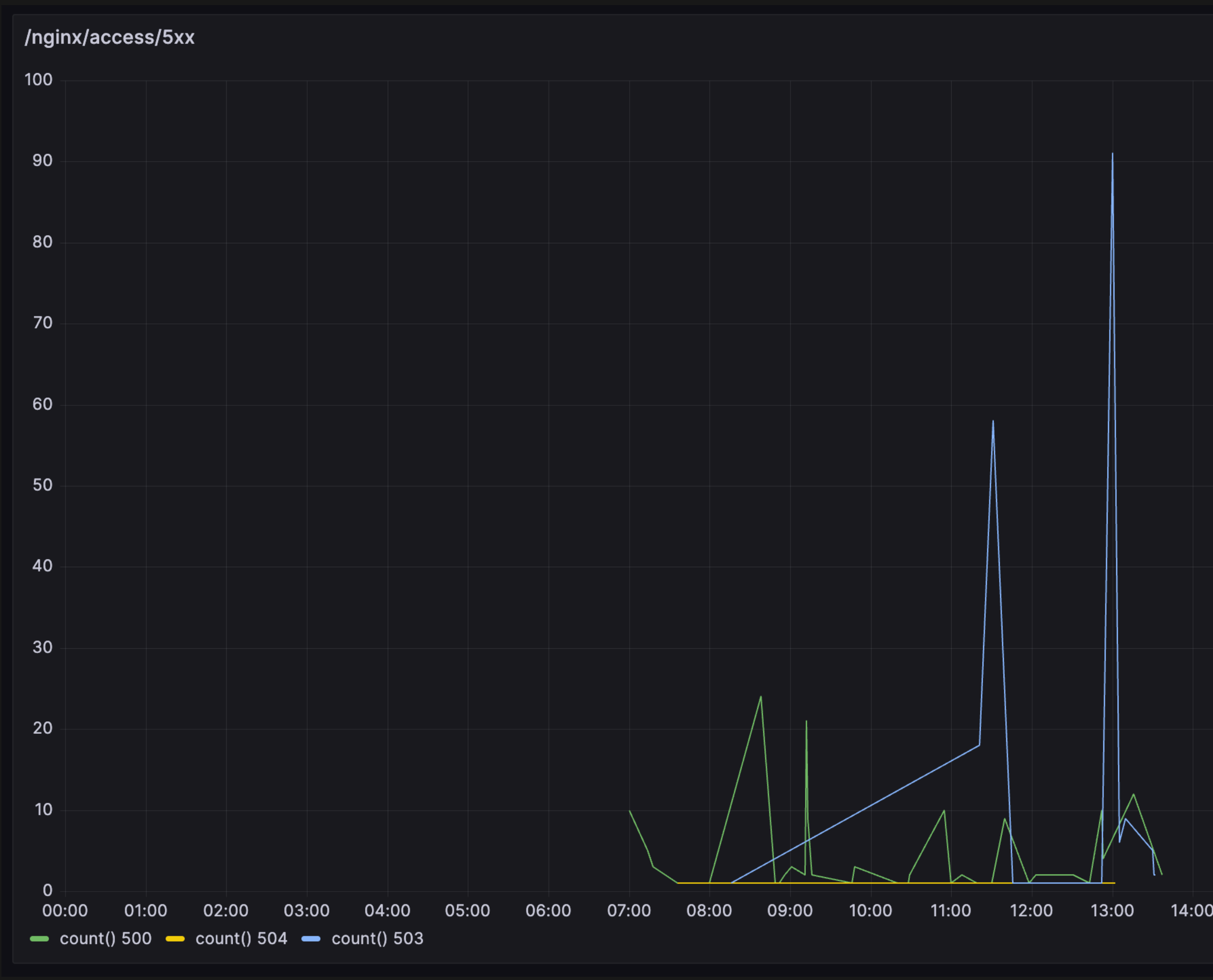
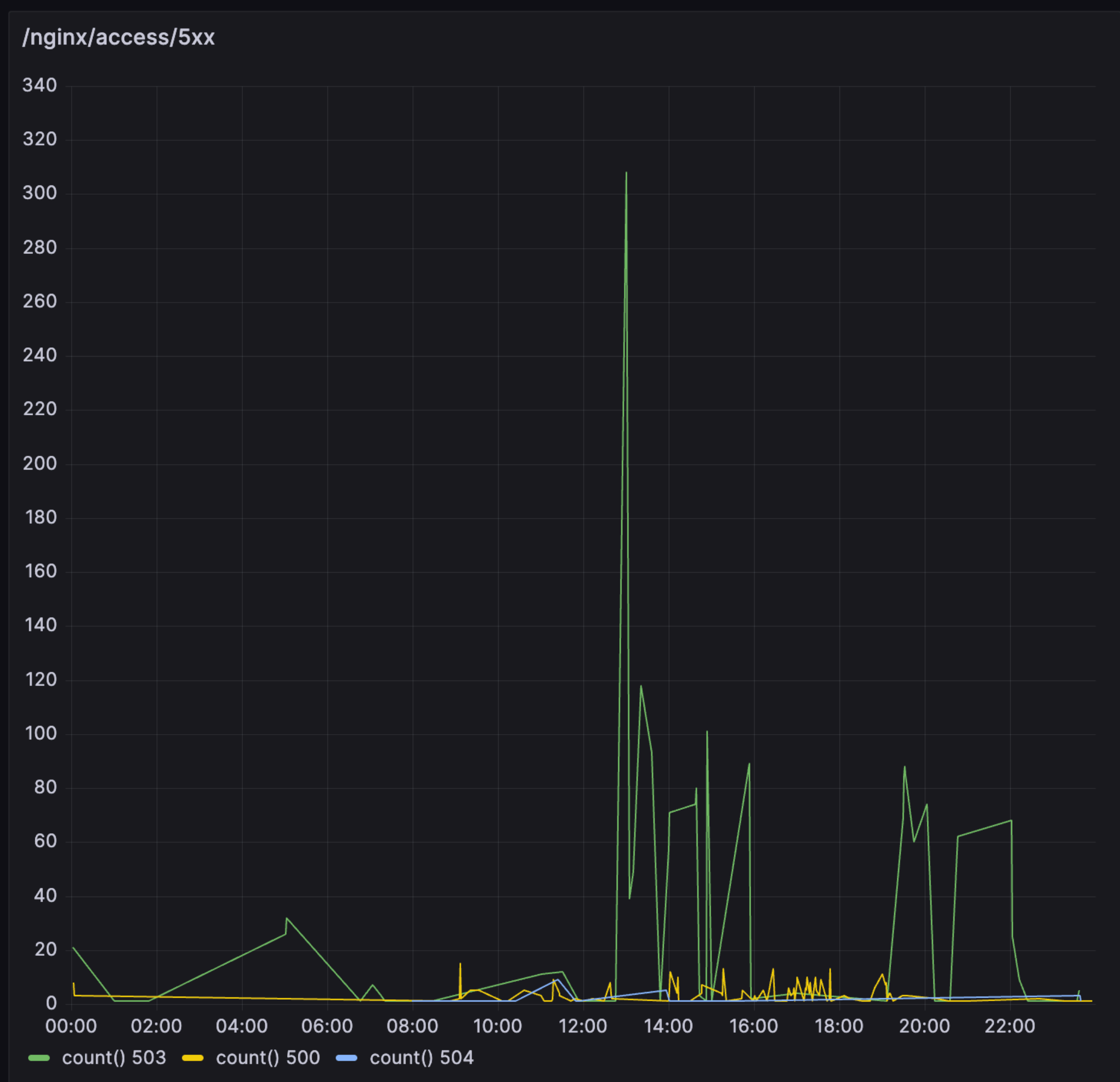
Optimization #7: Kong Retry

Rolling Update with Kong

- Kong retry: credit to @qinyuguang
- By default:
 - Connect timeout
 - 5xx

```
apiVersion: v1
kind: Service
metadata:
  name: server-service
  namespace: testing
  labels:
    version: v1
  annotations:
    konghq.com/retries: "5"
spec:
  selector:
    app: test-server
  ports:
    - port: 8080
      targetPort: 8080
      name: http
```

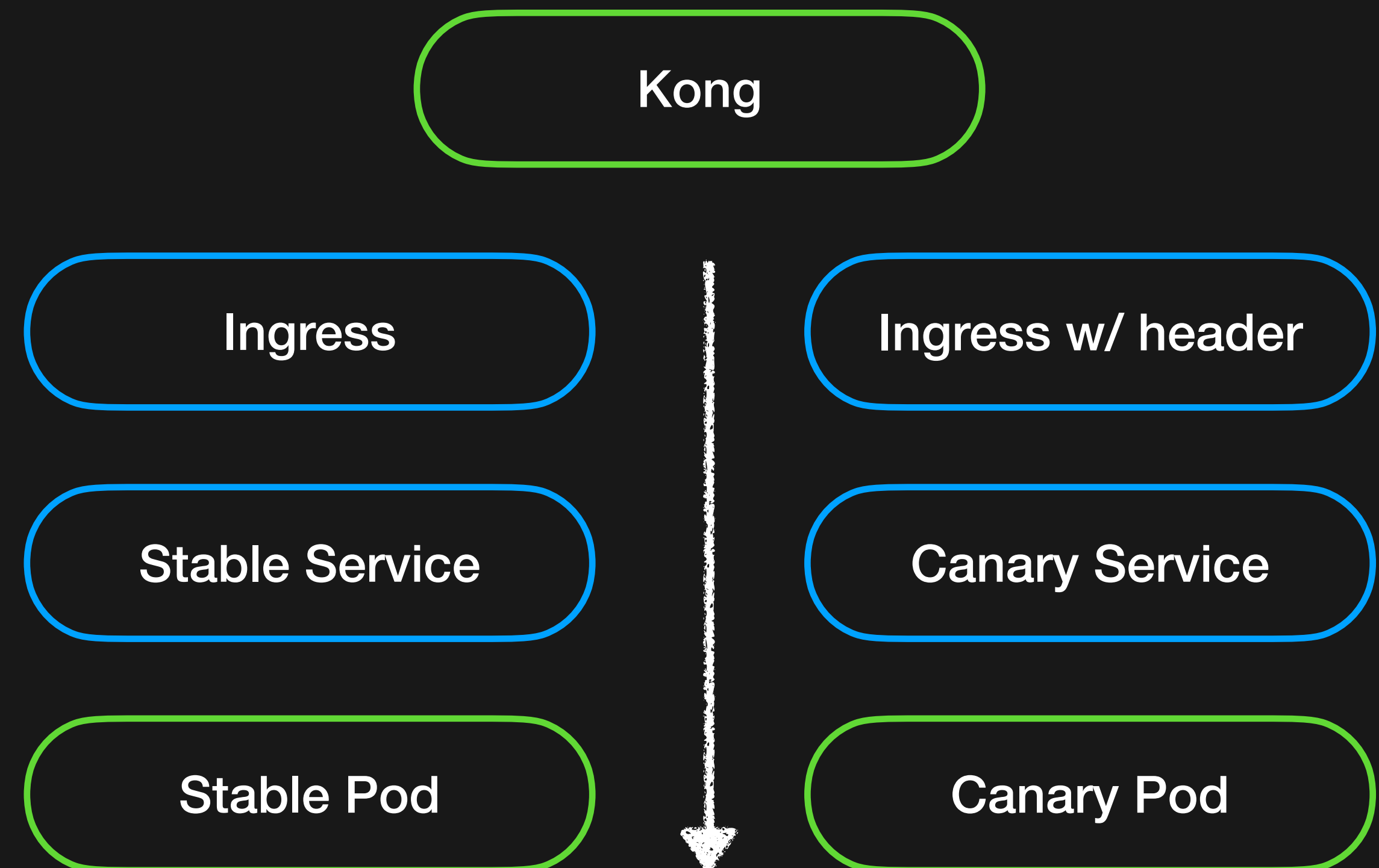

Optimization #7: Kong Retry Effect



Optimization #9: Kong Related Canary Deployment

Rolling Update with Kong

- What have we done wrong ?
 - Ingress -> Service (stage: all)
- Instead of point Ingress to “stage=all” service, point Ingress to “stage=stable”
- Add another Ingress with header for canary



Optimization #9: Kong Related Canary Deployment

Use different header for testing

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: server-service1-kong1
  namespace: testing
  annotations:
    kubernetes.io/ingress.class: "kong1"
    konghq.com/protocols: "http"
spec:
  rules:
    - host: server-service.testing.ntrnl-eks-fan.meican
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: server-service1
                port:
                  name: http
```

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: server-service2-kong1
  namespace: testing
  annotations:
    kubernetes.io/ingress.class: "kong1"
    konghq.com/protocols: "http"
    konghq.com/headers.version: v2
spec:
  rules:
    - host: server-service.testing.ntrnl-eks-fan.meican
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: server-service2
                port:
                  name: http
```

Diff

Optimization #9: Kong Related Canary Deployment Workaround

- `curl http://server-service.testing.ntrnl-eks-fan.meican:80/call`
- `curl -H 'version: v2' http://server-service.testing.ntrnl-eks-fan.meican:80/call`
- Require manual operation, incompatible with ArgoCD Rollout
- Awaiting GatewayAPI implementation

Takaways

- Prefer intra cluster address: `.svc.cluster.local`
- Pay attention to ndots to save DNS queries
- Explained Istio mechanism
 - Envoy takes over the traffic
 - Virtual Service
 - Destination Rule
- Explained Kong mechanism
 - Upstream-service: Local kube-proxy controlled IPtables
 - Pre-stop hook
 - Retries
 - Canary Deployment

References

- <https://planetmeican.atlassian.net/wiki/spaces/DEV/pages/1145208833/Kong+Retries>

Fin.

Question ?